

Animação Física Simplificada em Web

Trabalho de Conclusão de Curso

Diego Vasconcelos Schardosim de Matos

Universidade Federal do Rio de Janeiro
Instituto de Computação

15 de maio de 2026



Sumário

- 1 Introdução
- 2 Animação Baseada em Física
- 3 Detecção de Colisões
- 4 Resposta a Colisões
- 5 Otimizações
- 6 Resultados
- 7 Conclusão

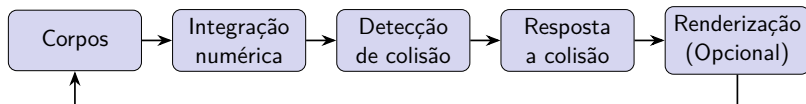


Motivação

- Jogos
- Simulações
- Robótica
- Desafio educacional e técnico



Pipeline de Animação Física



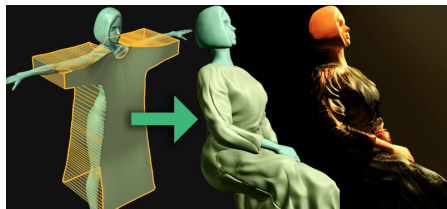
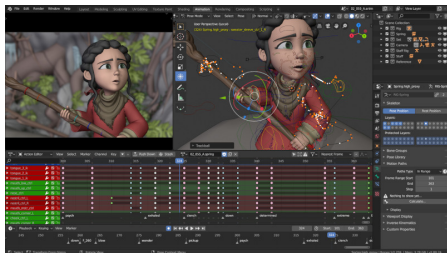
Motores de referência: Havok, PhysX, Bullet, Box2D. Todos inspiram a proposta Web deste trabalho.

Sumário

- 1 Introdução
- 2 Animação Baseada em Física
- 3 Detecção de Colisões
- 4 Resposta a Colisões
- 5 Otimizações
- 6 Resultados
- 7 Conclusão



Keyframe vs. Baseada em Física

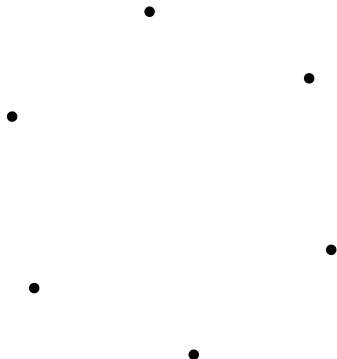


Objetivo: **plausibilidade visual**, não precisão científica.

Partículas

Não simulamos o objeto como um todo, simulamos cada **partícula** que o compõe.

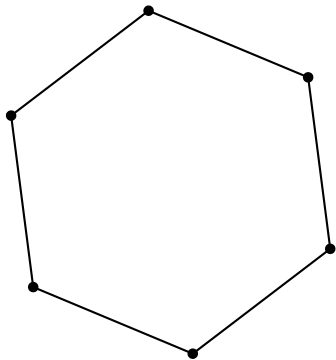
- Posição $\vec{x}$
- Forças aplicadas (ex: gravidade $\vec{g}$)



Formas Geométricas

Um objeto é um conjunto de **partículas** associado a uma **forma geométrica**.

Neste trabalho: apenas **formas convexas**.

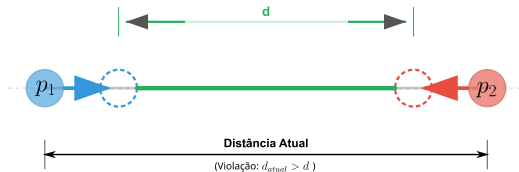


Restrições Geométricas

$$|\vec{x}_i - \vec{x}_j| = d$$

Após cada passo,

partículas são **projetadas**
de volta.

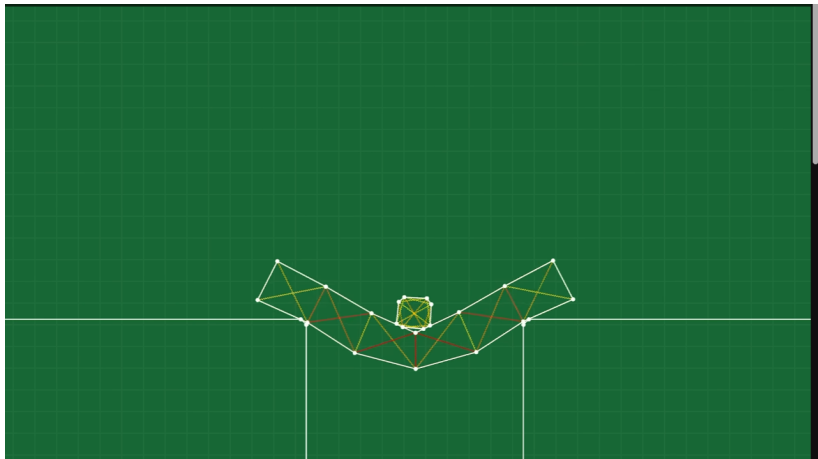


Resolvendo Restrições por Relaxamento

Resolver localmente e repetir ≈ 3
a 5 vezes converge o sistema
global.

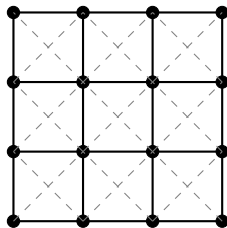
- 1: $\vec{\delta} \leftarrow \vec{x}_2 - \vec{x}_1$
- 2: $c \leftarrow \frac{\|\vec{\delta}\| - d}{\|\vec{\delta}\|}$
- 3: $\vec{x}_1 \leftarrow \vec{x}_1 - \epsilon \vec{\delta} c$
- 4: $\vec{x}_2 \leftarrow \vec{x}_2 + \epsilon \vec{\delta} c$

Representação dos Corpos



Estrutura Topológica do Tecido

- 1 **Estruturais**
horizontal/vertical
- 2 **Cisalhamento**
diagonal
- 3 **Flexão**
vizinho alternado



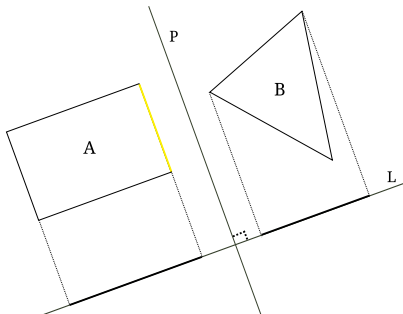
— Estrutural
- - - Cisalham.

Sumário

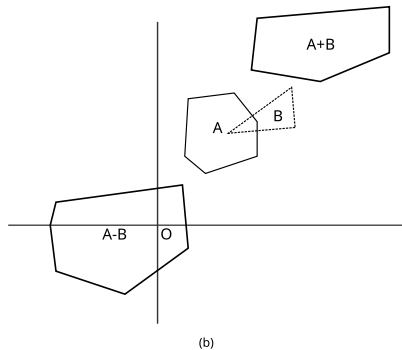
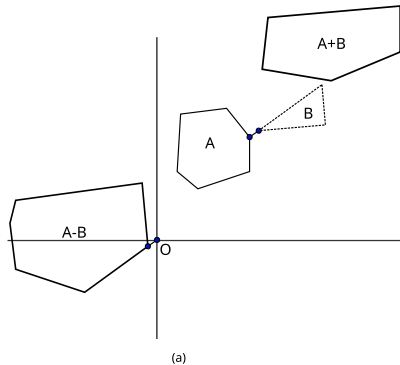
- 1 Introdução
- 2 Animação Baseada em Física
- 3 Detecção de Colisões**
- 4 Resposta a Colisões
- 5 Otimizações
- 6 Resultados
- 7 Conclusão

Teorema do Eixo Separador (SAT)

Projeções de A e B **não se sobrepõem** em algum eixo normal de aresta $\Rightarrow$ **sem colisão**.



Algoritmo de Gilbert-Johnson-Keerthi (GJK)



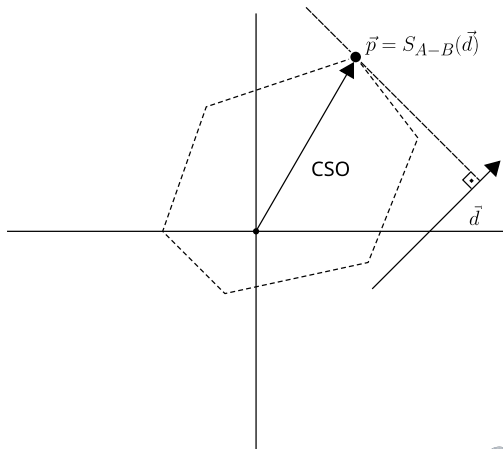
$$A - B = \{ \vec{x} - \vec{y} \mid \vec{x} \in A, \vec{y} \in B \}$$

$$A \cap B \neq \emptyset \iff \mathbf{0} \in A - B$$

Função Suporte

$$S_A(\vec{d}) = \arg \max_{\vec{a} \in A} (\vec{d} \cdot \vec{a})$$

$$S_{A-B}(\vec{d}) = S_A(\vec{d}) - S_B(-\vec{d})$$



GJK — Construção Iterativa do Simplex

A cada passo adiciona

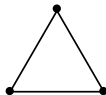
$$\vec{s} = S_{A-B}(\vec{d}):$$

- $\vec{s} \cdot \vec{d} \leq 0$

$\Rightarrow$ **sem colisão**

- Origem dentro do simplex

$\Rightarrow$ **colisão**

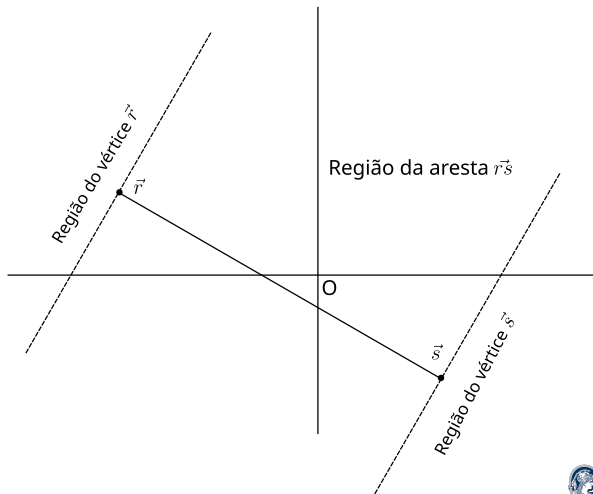


Subalgoritmo de Ericson — 1-simplex

Três regiões do segmento RS :

- Região de R
- Região de S
- Região da aresta

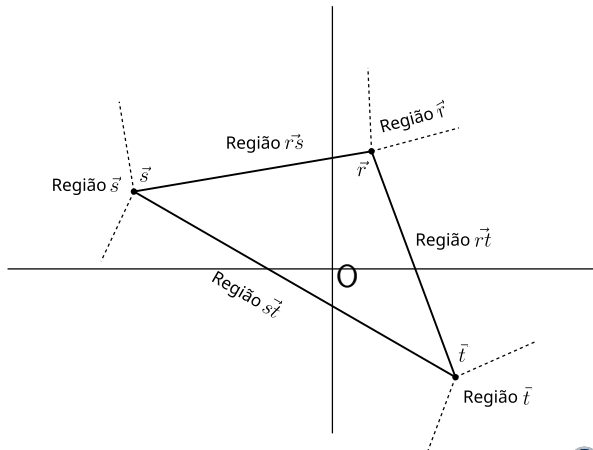
Regiões de vértice
 $\Rightarrow$ **sem colisão.**



Subalgoritmo de Ericson — 2-simplex

7 regiões $\rightarrow$ verificar
3:

- Aresta $\vec{rs}$
- Aresta $\vec{rt}$
- Interior $\triangle RST$
 $\Rightarrow$ **colisão**



Sumário

- 1 Introdução
- 2 Animação Baseada em Física
- 3 Detecção de Colisões
- 4 Resposta a Colisões**
- 5 Otimizações
- 6 Resultados
- 7 Conclusão



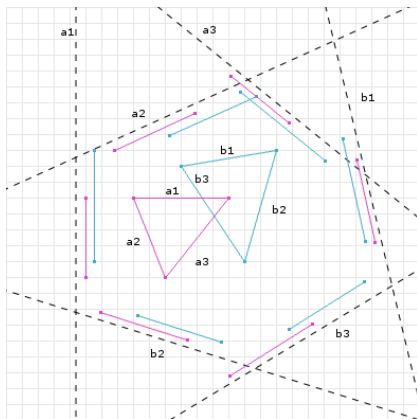
Pontos de Contato

Inferidos a partir da região de intersecção entre dois objetos.

Manifold de Colisão

- Normal $\hat{n}$
- Profundidade d
- Pontos de contato

Testar **todos** os eixos.
Guardar o de **menor**
sobreposição $\Rightarrow \hat{n}$ e δ .

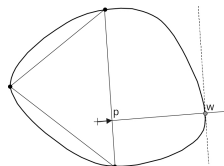


EPA — Expansão de Politopos

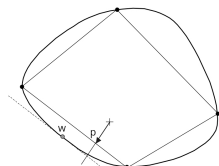
GJK detecta colisão mas **não** fornece $\hat{n}/\delta$.

EPA parte do simplex final do GJK e **expande** até:

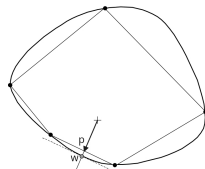
$$\|\vec{w}\| - \|\vec{p}\| < \epsilon$$



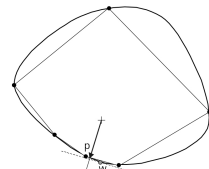
(a) $k = 0$



(b) $k = 1$

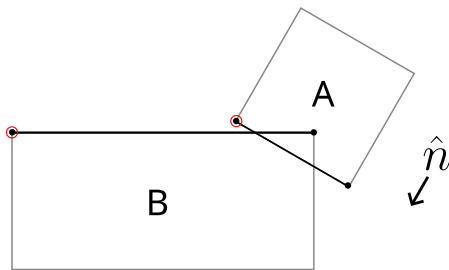


(c) $k = 2$

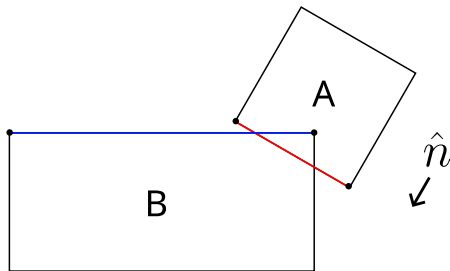


(d) $k = 3$

Cálculo dos Pontos de Contato



Arestas candidatas



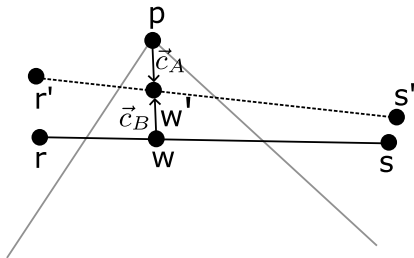
Referência (vermelho) e incidente (azul)

Processo de Separação — Vértice-Aresta

$$\vec{c} = d \hat{n}, \quad \alpha = \frac{(p-r) \cdot (s-r)}{\|s-r\|^2}$$

$$p' = p + \vec{c}_A$$

$$r' = r + (1-\alpha) \vec{c}_B, \quad s' = s + \alpha \vec{c}_B$$



Integração de Verlet

$$\vec{x}_{t+\Delta t} = 2 \vec{x}_t - \vec{x}_{t-\Delta t} + \vec{a}_t \Delta t^2$$

$$\vec{v}_t \approx \frac{\vec{x}_{t+\Delta t} - \vec{x}_{t-\Delta t}}{2 \Delta t}$$

Vantagens

- Estável sob restrições
- Velocidade implícita

Sumário

- 1 Introdução
- 2 Animação Baseada em Física
- 3 Detecção de Colisões
- 4 Resposta a Colisões
- 5 Otimizações**
- 6 Resultados
- 7 Conclusão

Filtragem e Refinamento

Fase de Filtragem (Broad Phase)

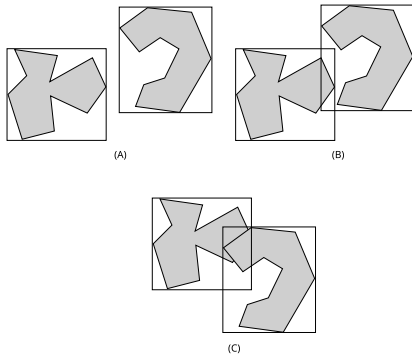
Descartar pares impossíveis
usando objetos delimitadores.

Fase de Refinamento (Narrow Phase)

Usar algoritmos sofisticados como
SAT / GJK+EPA apenas nos
pares candidatos



- Teste de interseção de baixo custo
- Ajuste preciso
- Cálculo econômico
- Fácil de girar e transformar
- Baixo consumo de memória

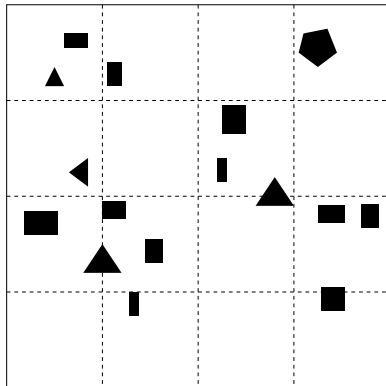


Filtragem com Grade Uniforme

$$i = \left\lfloor \frac{x}{C} \right\rfloor, \quad j = \left\lfloor \frac{y}{C} \right\rfloor$$

$$Key = i + j \times n_{col}$$

Candidatos $\Leftrightarrow$ mesma célula.

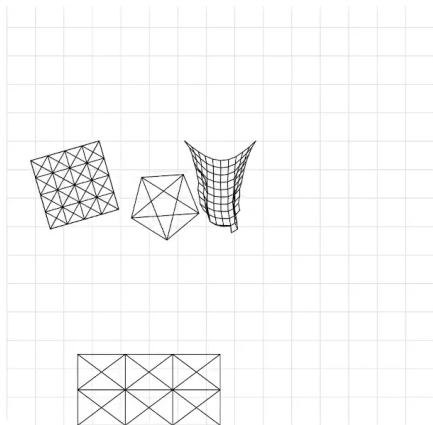
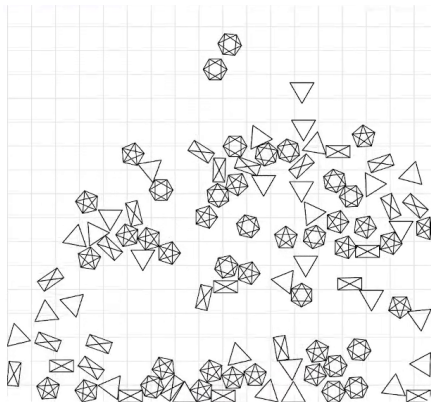


Sumário

- 1 Introdução
- 2 Animação Baseada em Física
- 3 Detecção de Colisões
- 4 Resposta a Colisões
- 5 Otimizações
- 6 Resultados**
- 7 Conclusão



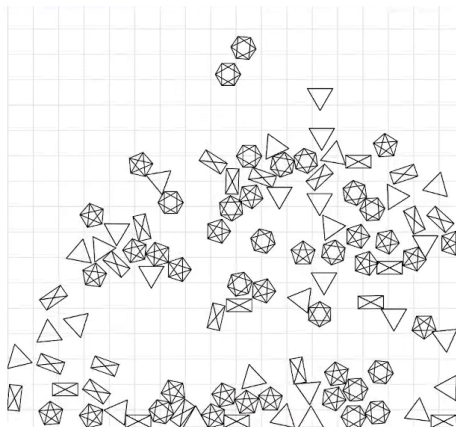
Ambiente de Visualização



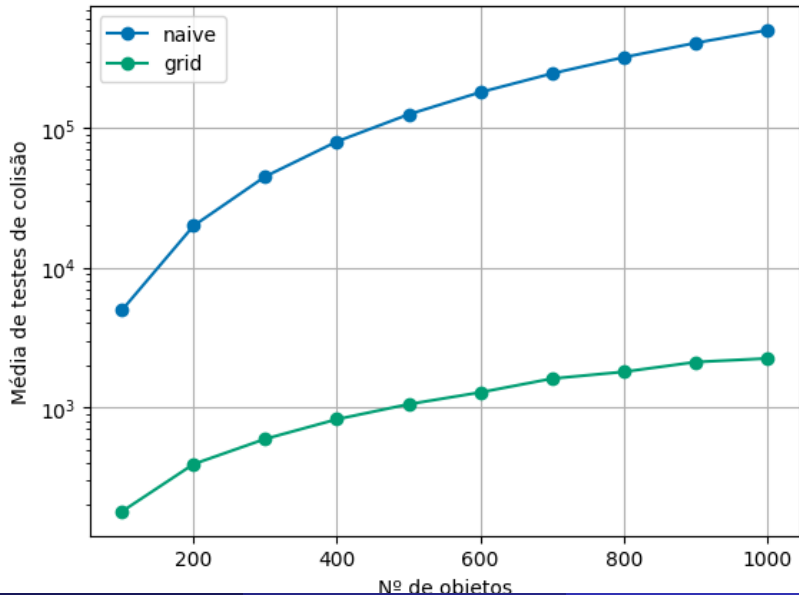
biblioteca typescript · modos SAT e GJK/EPA · p5.js

Cenários de Teste

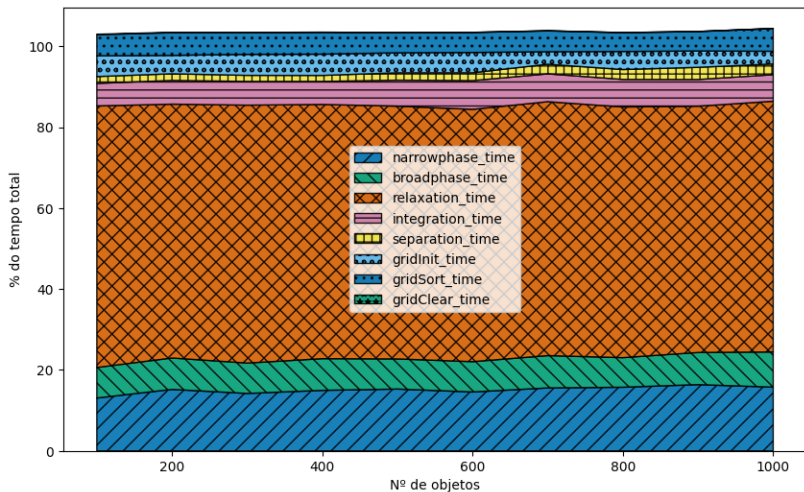
- 100 a 1000 objetos (passo 100)
- 180 quadros por config.
- Sem renderização
- *Poisson Disk Sampling*
- Ryzen 5 5500U, 24GB RAM DDR4, Ubuntu 24.04LTS



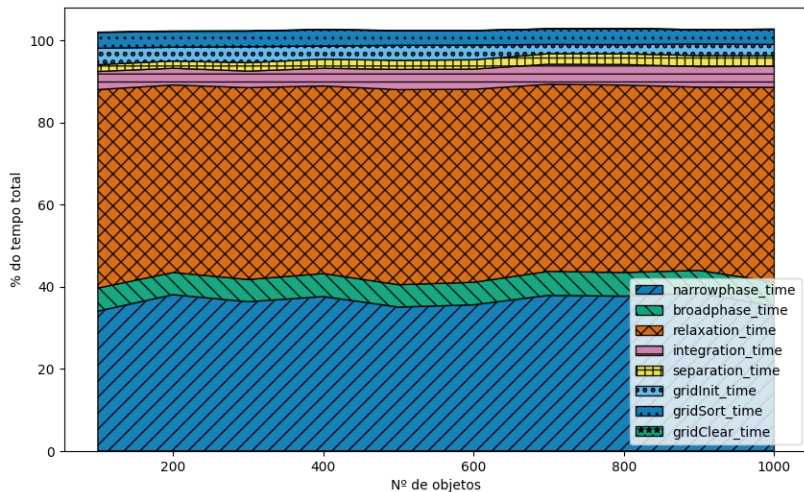
Impacto da Filtragem



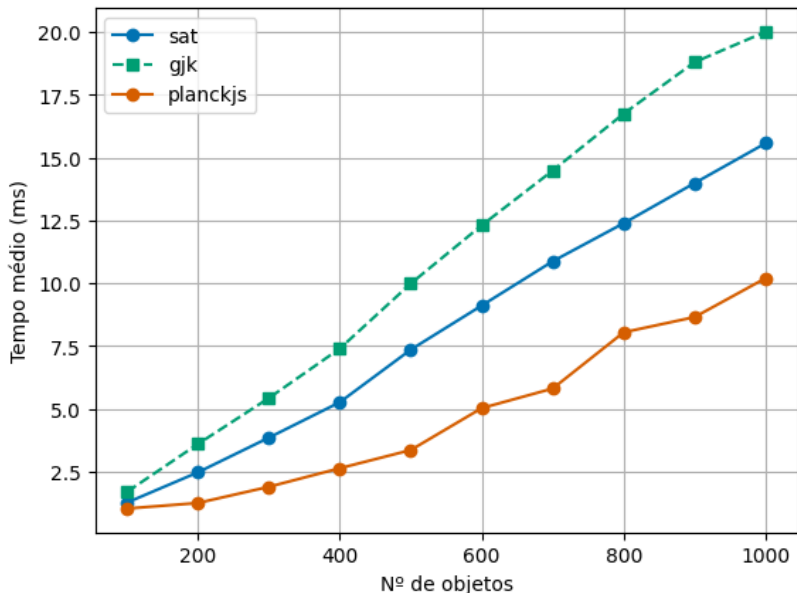
Tempo por Etapa — SAT



Tempo por Etapa — GJK/EPA



Comparação com *Planck.js*



Sumário

- 1 Introdução
- 2 Animação Baseada em Física
- 3 Detecção de Colisões
- 4 Resposta a Colisões
- 5 Otimizações
- 6 Resultados
- 7 Conclusão**

Conclusões e Trabalhos Futuros

Alcançado

- Motor funcional em TypeScript
- SAT, GJK+EPA implementados
- Separação geométrica
- Filtragem com Grade uniforme
- Código aberto e reproduzível

Trabalhos Futuros

- Extensão para 3D
- Paralelização com Webworks e WebGPU
- BVH / Quadtree na filtragem

Referências



T. Jakobsen. Advanced character physics. *GDC*, 2001.



C. Ericson. *Real-Time Collision Detection*. Morgan Kaufmann, 2004.



G. van den Bergen. Proximity queries and penetration depth computation on 3D game objects. *GDC*, 2001.



C. Murtori. Implementing GJK.
https://caseymurtori.com/blog_0003, 2006.



Dyn4j. Contact points using clipping.
<https://dyn4j.org/2011/11/contact-points-using-clipping/>, 2011.



A. Shakiba. *Planck.js*. <https://piqnt.com/planck.js/>.

Obrigado!

`diegovsm@ic.ufrj.br`

`https:`

`//github.com/devdiegomatos/simplified-physics-animation-web`